

Module M05.C04 — pandas pour transformer : `pipe`, `assign`, `query`, `merge`, et le notebook reproductible

Analyse de données appliquée & Business Intelligence
M05_C04_pandas_pour_transformer.md · document de production

Table des matières

MODULE M05.C04 — PANDAS POUR TRANSFORMER : PIPE, ASSIGN, QUERY, MERGE, ET LE NOTEBOOK REPRODUCTIBLE	3
1. Objectifs du chapitre	3
2. Pourquoi cette notion est importante	3
3. Explication simple	4
4. Vocabulaire essentiel	4
5. Cours approfondi — la table des 10 opérations en pandas	6
5.1 Étape 0 — Importer le CSV	6
5.2 Étape 1 — Sélectionner / renommer les colonnes	6
5.3 Étape 2 — Convertir le type (avec compteur d'échec)	6
5.4 Étape 3 — Nettoyer le texte	7
5.5 Étape 4 — Filtrer les lignes (la coupe à la date)	7
5.6 Étape 5 — Trier	7
5.7 Étape 6 — Dédoublonner (clé métier)	7
5.8 Étape 7 — Colonne calculée (la transposition jour/mois)	8
5.9 Étape 8 — Fusionner (la jointure aux remises)	8
5.10 Étape 9 — Agréger (CA par catégorie)	9
5.11 Étape 10 — Pivoter / dépivoter	9
5.12 Étape 11 — Exporter la table propre	9
5.13 Le notebook reproductible (.ipynb)	10
5.14 Le diagnostic en 4 questions (le « Health Check » pandas)	10
6. Exemple concret — le notebook mars2025.ipynb complet	10
7. Démonstration pas à pas — le verdict : un fichier, trois moteurs, une table	12
7.1 La sérialisation canonique en pandas	12
7.2 Le verdict du projet M05.P (la mesure d'ouverture)	12
7.3 La rejouabilité : avril et mai 2025	13
8. Erreurs fréquentes	13
9. Bonnes pratiques professionnelles	14
9.1 Migration pandas 2.x → 3.x	14
10. Exercice guidé — le notebook avril2025.ipynb paramétré (45 min, /10)	14
11. Exercices autonomes	15
12. Correction détaillée	15
13. Mini-projet M05.P4 — « Le notebook pandas du projet M05.P » (1 h 30)	16
14. Boîte à outils du chapitre	17
15. Résumé du chapitre	17
16. À retenir	17
17. Évaluation formative (auto-correction, 8 min)	18

Module M05.C04 — pandas pour transformer : pipe, assign, query, merge, et le notebook reproductible

Outil de ce chapitre : pandas 2,2,3 (Python 3,13), sur le fil rouge mars 2025 du socle (6 884 lignes). Durée indicative : 7 h. Niveau : N2.

L'idée du chapitre. pandas n'est pas un outil de **feuille de calcul** : c'est un outil de **transformation de tables**. Les 10 opérations du C01 ont une forme pandas — `df.assign`, `df.query`, `df.sort_values`, `df.drop_duplicates`, `df.merge`, `df.groupby`, `df.pivot_table`, `df.melt`. Le chapitre installe la **chaîne de transformations** (le pipe), les **méthodes de dédoublonnage et jointure** (les mêmes que SQL et Power Query, en syntaxe pandas), et le **notebook reproductible** : un `.ipynb` qui, ré-exécuté du début à la fin, produit la même table que les deux autres moteurs (le verdict du C01, empreinte sha256 identique).

Base de travail — le fil rouge mars 2025, mesuré en pandas 2,2,3. Coupe à la date 2025-03-01 → 2025-03-31 : **6 884 lignes · total montant_ttc 464 096 003 FCFA** (mesure `m05_filrouge_mars_total_ttc`). Le notebook complet du §6 produit **la même table** que la requête Power Query de C02 et la requête SQL de C03 — c'est le **verdict** : trois moteurs, une table, une empreinte sha256 (mesure `m05p Empreinte_sha256` = 7cce2d0c6253f739-faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **importer** un CSV avec `pd.read_csv(..., dtype=str, keep_default_na=False)` (la règle du module : toujours importer en `str`, puis convertir) ;
- **chaîner** des transformations avec `pipe`, `assign`, `query` ;
- **convertir** avec `pd.to_numeric(..., errors="coerce")` + compteur `isna().sum()` ;
- **dédoubler** avec `drop_duplicates(..., keep="first")` sur la clé métier 5-colonnes ;
- **réparer** les dates transposées avec une affectation par masque booléen ;
- **fusionner** avec `merge(..., on=[...], how="left")` sur la clé complète ;
- **agréger** avec `groupby` + `agg`, **pivoter** avec `pivot_table`, **dépivoter** avec `melt` ;
- **exporter** en CSV ou Parquet avec `to_csv` ou `to_parquet` ;
- **vérifier le verdict** par empreinte sha256 — trois moteurs, une table.

2. Pourquoi cette notion est importante

- pandas est l'outil de **transformation en mémoire** : il est 30 × plus lent que DuckDB sur ce fichier (3,1 s vs 0,4 s sur 6 884 lignes), mais il est **plus riche en méthodes** (`pipe`, `assign`, `query`, `melt`, `pivot_table`, `merge`, `groupby`) et **plus adapté à l'analyse exploratoire** (statistiques descriptives, `groupby` complexes). C'est l'outil de l'analyste, par opposition à l'outil de l'ingénieur (DuckDB) ou du bureautique (Power Query).
- Le **compteur d'échec** (`isna().sum()` après `pd.to_numeric`) est la première discipline. pandas `read_csv` est tolérant : il devine `montant_ttc` en `object` (texte) et **avale les 1 710 montants texte sans un mot**. Sans compteur, on additionne du texte et on perd 1,5 % du CA sans s'en apercevoir.

- La **chaîne de transformations** (`pipe`, `assign`) est ce qui rend le code pandas **lisible et reproductible** : chaque étape est une fonction, chaque étape a un nom, chaque étape peut être testée indépendamment. C'est l'équivalent des étapes Power Query et des `WITH SQL`.
- Le **notebook reproductible** (`.ipynb`) est l'unité de livraison : un fichier qui, ré-exécuté du début à la fin, produit la même table. Sans cette garantie, le notebook est **statique** : il produit une table, à un moment donné, et le collègue ne peut pas la reproduire.
- pandas **diffère de SQL** sur trois points : (a) l'ordre d'évaluation des filtres (pandas est paresseux, SQL est immédiat) ; (b) le typage strict (pandas devine les types à l'import, SQL **impose** les types dans le `CREATE TABLE AS`) ; (c) la gestion des `NaN` (pandas met `NaN` pour les nombres, SQL met `NULL` pour tous les types). Ces différences sont la source de la plupart des bugs pandas-vs-SQL.
- Enfin, **pandas 2,2,3 dans l'atelier, 3.x dans le manuel** : la syntaxe est largement compatible, mais certaines méthodes changent de signature entre 2.x et 3.x (ex. `inplace` est déprécié). Le chapitre signale les divergences (encadré « Migration »).

3. Explication simple

On ne programme pas dans un notebook : on **chaîne des transformations** sur un `DataFrame`. Chaque transformation prend un `DataFrame`, en rend un autre, et le résultat est affecté à une variable. La variable finale est la table propre, exportée en CSV ou Parquet.

Le notebook complet du fil rouge (mars 2025) a **8 cellules** dans C04 : import → conversion → coupe → tri → dédoublonnage → réparation → enrichissement → agrégation/export. C'est **plus court** que la requête SQL de C03 (11 tables) — mais **exécutable** dans l'atelier (règle 8 du §3 du plan M05).

La **règle d'or** du pandas : **chaque transformation a un nom, en toutes lettres, et est appliquée par `assign` ou `pipe`**. Une transformation inplace (`df["col"] = df["col"].fillna(...)`) est illisible et **non reproductible**.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
DataFrame DataFrame	— Structure tabulaire à deux dimensions, étiquetée par lignes et colonnes. C'est l'équivalent d'une table SQL ou d'une requête Power Query.	Confondre <code>df</code> (le <code>DataFrame</code>) et <code>df.copy()</code> (la copie). Sans <code>.copy()</code> , les modifications inplace affectent l'original — c'est l'erreur classique du débutant.
Série — Series	Une colonne d'un <code>DataFrame</code> . C'est l'équivalent d'un vecteur numpy.	Appliquer une fonction pandas à une Série sans le vectoriser : <code>df["col"].apply(f)</code> est 100 × plus lent que <code>df["col"].str.replace(...)</code> .
dtype — dtype	Le type d'une colonne : <code>int64</code> , <code>float64</code> , <code>object</code> (texte), <code>datetime64[ns]</code> , <code>Int64</code> (nullable), etc.	Importer en <code>int64</code> par défaut : les <code>id_vente = 0</code> (ventes comptoir) sont mal lus, et les montants texte deviennent <code>int64</code> avec des valeurs aberrantes.
pipe — pipe	Méthode qui applique une fonction à un <code>DataFrame</code> et renvoie le résultat. <code>df.pipe(f, arg1, arg2)</code> est équivalent à <code>f(df, arg1, arg2)</code> .	Oublier le <code>return</code> dans la fonction passée à <code>pipe</code> : pandas renvoie <code>None</code> , et l'erreur est obscure.
assign — assign	Méthode qui ajoute ou remplace des colonnes sans modifier le <code>DataFrame</code> original. <code>df.assign(ca_ht=...)</code> renvoie un nouveau <code>DataFrame</code> .	Confondre <code>assign</code> et l'affectation inplace (<code>df["ca_ht"] = ...</code>) : l'affectation modifie l'original et brise la reproductibilité.

Français — English	Définition simple	Piège à éviter
query — query	Méthode qui filtre les lignes avec une expression chaîne : <code>df.query("montant_ttc > 1000")</code> . C'est plus lisible que le filtrage booléen.	Oublier les guillemets autour des chaînes : <code>df.query("ville == 'Ouaga'")</code> est valide, <code>df.query("ville == Ouaga")</code> lève une erreur.
drop_duplicates — drop_duplicates	Méthode qui retire les doublons sur une clé. <code>df.drop_duplicates(["col1", "col2"], keep="first")</code> garde la première occurrence.	Utiliser <code>keep=False</code> : toutes les occurrences sont retirées, y compris les répétitions intraticket légitimes. La règle est toujours <code>keep="first"</code> sauf cas exceptionnel.
merge — merge	Méthode qui joint deux DataFrames sur une clé. <code>df.merge(other, on=["col1", "col2"], how="left")</code> est l'équivalent du LEFT JOIN SQL.	Joindre sur une clé partielle (<code>on="id_client"</code>) au lieu de la clé complète (<code>on=["id_client", "annee", "mois"]</code>) : c'est l'erreur du module (3 978 lignes au lieu de 101 sur mars).
groupby groupby	Méthode qui regroupe les lignes par clé et applique une agrégation. <code>df.groupby(["cat"], as_index=False)["ca"].sum()</code> est l'équivalent du GROUP BY SQL.	Oublier <code>as_index=False</code> : la clé de regroupement devient l'index, et l'accès aux colonnes est bizarre (<code>df.loc["cat_value"]</code>).
pivot_table pivot_table	Méthode qui pivote une table : <code>df.pivot_table(index="...", columns="...", values="...", aggfunc="sum", fill_value=0)</code> .	Pivoter sur une colonne à 1 000 valeurs distinctes : pandas crée 1 000 colonnes, et la RAM explose.
melt — melt	Méthode qui dépivot : <code>df.melt(id_vars=["mois"], value_vars=["P1", "P2"], var_name="produit", value_name="ca")</code> est l'inverse du pivot.	Dépivoter une colonne d'identifiant (<code>id_vente</code>) : chaque id est dupliqué. La règle est dépivoter sur les colonnes de mesure.
read_csv read_csv	Fonction qui lit un CSV. <code>pd.read_csv(path, dtype=str, keep_default_na=False, encoding="utf-8-sig")</code> est la signature du module.	Oublier <code>keep_default_na=False</code> : pandas transforme les chaînes "", NA, N/A en NaN, et le compteur d'échec est faussé.
to_csv to_csv to_parquet to_parquet	Méthodes qui exportent un DataFrame. <code>df.to_csv(path, index=False)</code> exporte en CSV ; <code>df.to_parquet(path)</code> exporte en Parquet.	Oublier <code>index=False</code> dans <code>to_csv</code> : l'index du DataFrame devient la première colonne du CSV, et le verdict diffère.
Graine — random seed	La graine aléatoire (<code>np.random.seed(42)</code> ou <code>random_state=42</code>) qui rend les opérations aléatoires reproductibles.	Oublier la graine sur un <code>sample()</code> : deux exécutions donnent deux mesures différentes.

Définition. Le **préambule controle_python.py** est le bloc de code exécuté en premier dans chaque cellule du notebook de référence. Il charge les variables partagées (`pd`, `np`, `stats` (scipy), `RACINE`, `df`, `pop`). Les variables sont partagées entre les cellules, mais **chaque cellule doit être autonome** (ré-exécutable isolément) — c'est la règle 8 du §3 du plan M05.

Définition. Le `.loc[mask, "col"]` est l'opérateur pandas d'affectation par masque booléen : `df.loc[mask, "col"] = nouvelle_valeur` modifie uniquement les lignes où `mask` est `True`. C'est l'écriture pandas de la mise à jour conditionnelle, et c'est vectorisée (rapide) — à l'opposé de `df["col"][mask] = ...` qui lève un `SettingWithCopyWarning`.

Définition. Le **SettingWithCopyWarning** est un avertissement pandas qui signale qu'une modification peut ne pas être appliquée (parce qu'on travaille sur une vue du DataFrame, pas une copie). La règle est toujours `df = df.copy()` après un filtrage, et toujours `df.loc[mask, "col"] = ...` pour les modifications conditionnelles.

Définition. Le **format Parquet** est un format de stockage tabulaire **typé et compressé** (5-10 × plus compact que CSV). pandas écrit et lit le Parquet nativement (`to_parquet`, `read_parquet`). La règle du module est toujours exporter en Parquet pour les tables > 100 000 lignes, et en CSV pour les tables plus petites.

5. Cours approfondi — la table des 10 opérations en pandas

Cette section reprend la table du C01 (§5) et l'applique **exclusivement à pandas 2,2,3**, avec un détail suffisant pour qu'un apprenant puisse reproduire le notebook sans aide. Les blocs Python publiés ici sont **exécutés** dans l'atelier — chaque chiffre cité est mesuré sur le fichier livré.

5.1 Étape 0 — Importer le CSV

```
import pandas as pd

SRC = "01_socle_donnees/data/brut/ventes_brutes.csv"
df = pd.read_csv(SRC, dtype=str, keep_default_na=False, encoding="utf-8-sig")

print("lignes importées :", len(df)) # 243 360
print("colonnes :", df.columns.tolist()) # 20 colonnes
```

- **`dtype=str`** : on force l'import en texte. C'est la règle du module — on convertit ensuite colonne par colonne, en contrôlant l'échec.
- **`keep_default_na=False`** : pandas ne transforme pas `" "` en `NaN`. Sans ça, le compteur d'échec de conversion est faussé.
- **`encoding="utf-8-sig"`** : on retire le BOM (les 3 octets `EF BB BF` au début). Sans ça, la première colonne s'appelle `ï»¿id_vente`.

5.2 Étape 1 — Sélectionner / renommer les colonnes

```
df_v1 = df[[
    "id_vente", "date_vente", "id_ticket", "id_produit", "quantite",
    "montant_ttc", "taux_remise", "id_client", "annee", "mois", "heure"
]].copy() # .copy() pour éviter les modifications inplace
```

- Le **`.copy()`** est crucial : sans lui, `df_v1` est une vue de `df`, et toute modification de `df_v1` modifie `df`.
- Le **renommage** se fait par `df.rename(columns={"ancien": "nouveau"})` — à n'utiliser que pour des raisons métier.

5.3 Étape 2 — Convertir le type (avec compteur d'échec)

```
# Conversion de montant_ttc
df_v2 = df_v1.assign(
    montant_ttc_n=pd.to_numeric(
        df_v1["montant_ttc"].str.replace(" FCFA", "", regex=False).str.replace(" ", "",
        regex=False),
        errors="coerce",
    )
)

# Compteur d'échec
nb_cellules_vides = (df_v2["montant_ttc"].str.strip() == "").sum()
nb_echec_conversion = df_v2["montant_ttc_n"].isna().sum() - nb_cellules_vides
print("échec conversion :", nb_echec_conversion) # 114 sur mars 2025 (mesure
m05_filrouge_mars_montants_texte)
```

- **`pd.to_numeric(..., errors="coerce")`** : en cas d'échec, la cellule devient `NaN`. L'alternative `errors="ignore"` laisse la cellule en texte (et le compteur d'échec est 0).
- Le compteur d'échec est la **différence** entre le nombre de `NaN` et le nombre de cellules vides à l'origine. C'est la formule du module.

5.4 Étape 3 — Nettoyer le texte

```
# Nettoyage explicite de montant_ttc (retrait du suffixe " FCFA" et des espaces)
df_v3 = df_v2.assign(
    montant_ttc_clean=df_v2["montant_ttc"].str.replace(" FCFA", "", regex=False).str.strip(),
    montant_ttc_n2=pd.to_numeric(
        df_v2["montant_ttc"].str.replace(" FCFA", "", regex=False).str.replace(" ", "",
        regex=False),
        errors="coerce",
    ),
)

# Compteur après nettoyage
nb_echec_apres = df_v3["montant_ttc_n2"].isna().sum() - nb_cellules_vides
print("échec après nettoyage :", nb_echec_apres) # 0 sur mars (mesure
m05_filrouge_mars_montants_texte_apres_conversion)
```

- Le **nettoyage est volontairement en deux temps** : on trace la transformation (`montant_ttc_clean`) et on confirme la conversion (`montant_ttc_n2`).
- `str.strip()` retire les espaces en début/fin. `str.replace(" FCFA", "", regex=False)` retire le suffixe. `regex=False` est important : sans lui, pandas interprète `FCFA` comme une regex et le `F` est ambigu.

5.5 Étape 4 — Filtrer les lignes (la coupe à la date)

```
df_v4 = df_v3[df_v3["date_vente"].between("2025-03-01", "2025-03-31")].copy()
print("mars (v4) :", len(df_v4), "lignes") # 6 884 (mesure m05_filrouge_mars_lignes)
```

- `between` est inclusif aux deux bornes : `between("2025-03-01", "2025-03-31")` inclut le 1^{er} et le 31 mars.
- Le `.copy()` est crucial : sans lui, `df_v4` est une vue, et `df_v4["ca_ht"] = ...` lève un `SettingWithCopyWarning` (et peut modifier `df` silencieusement).

5.6 Étape 5 — Trier

```
df_v5 = df_v4.sort_values(["date_vente", "id_vente"], kind="mergesort").reset_index(drop=True)
```

- `kind="mergesort"` est le tri stable : deux lignes avec la même `date_vente` restent dans l'ordre de leur `id_vente`. C'est important pour le dédoublonnage (§5.7).
- `reset_index(drop=True)` réinitialise l'index : sans lui, l'index porte les positions d'origine, et `iloc[0]` ne pointe plus sur la première ligne de `df_v4` mais sur la première ligne de `df`.

5.7 Étape 6 — Dédoublonner (clé métier)

```
CLE = ["id_ticket", "id_produit", "quantite", "montant_ttc_n2", "heure"]
df_v6 = df_v5.drop_duplicates(CLE, keep="first").copy()
print("après dédoublonnage (v6) :", len(df_v6), "lignes") # 6 798 (6 884 - 86 ; mesure
m05_filrouge_mars_doublons)
```

- `keep="first"` garde la première occurrence selon l'ordre du tri (`mergesort` sur `date_vente` puis `id_vente`).
- La **clé 5-colonnes** est l'unique clé qui préserve les 6 répétitions intra-ticket légitimes (mesure `m05_repetitions_legitimes_brut`).

5.8 Étape 7 — Colonne calculée (la transposition jour/mois)

```
import numpy as np

# Détection des transpositions
d = pd.to_datetime(df_v6["date_vente"])
inco = d.dt.month != df_v6["mois"].astype(int)
print("dates transposées :", inco.sum()) # 23 sur mars (mesure
m05_filrouge_mars_dates_transposees)

# Réparation : échange jour et mois
df_v7 = df_v6.copy()
ds = d[inco]
df_v7.loc[inco, "date_vente"] = (
    ds.dt.year.astype(str) + "-" + ds.dt.day.astype(str).str.zfill(2)
    + "-" + ds.dt.month.astype(str).str.zfill(2)
)
```

- `d.dt.month != df_v6["mois"].astype(int)` : la détection. Si le mois de la date est différent du mois dans la colonne `mois`, c'est une transposition.
- L'affectation par masque booléen (`df_v7.loc[inco, "date_vente"] = ...`) est l'écriture pandas de la réparation. C'est **plus rapide** que `df_v7["date_vente"].apply(...)` (qui est une boucle Python).
- **Piège classique** : `df_v7["date_vente"] = ...` sans `.loc` lève un `SettingWithCopyWarning`, et la modification peut ne pas être appliquée.

5.9 Étape 8 — Fusionner (la jointure aux remises)

```
# Chargement et dédoublement des remises manuelles
remises_brutes = pd.read_excel("01_socle_donnees/data/brut/remises_manuelles.xlsx",
                               engine="openpyxl")
remises_dedup = remises_brutes.drop_duplicates(["id_client", "annee", "mois"], keep="first")

# Jointure à la clé complète
df_v8 = df_v7.merge(
    remises_dedup[["id_client", "annee", "mois", "remise_consensie_montant"]],
    on=["id_client", "annee", "mois"],
    how="left",
)
print("après jointure (v8) :", len(df_v8), "lignes") # 6 798 (inchangé : LEFT JOIN)
print("lignes enrichies :", df_v8["remise_consensie_montant"].notna().sum()) # 101 (mesure
m05_filrouge_mars_lignes_enrichies)

# La jointure naïve, pour comparer :
df_v8_naive = df_v7.merge(
    remises_dedup[["id_client", "annee", "mois", "remise_consensie_montant"]],
    on="id_client", # ← clé PARTIELLE
    how="inner",
)
print("jointure naïve :", len(df_v8_naive), "lignes") # 3 978 (mesure
m05_filrouge_mars_jointure_naive, ≈ 39 ×)
```

- La **clé complète** (`on=["id_client", "annee", "mois"]`) est l'unique protection contre la jointure naïve.
- `how="left"` garde toutes les lignes de `df_v7` (les 6 798), et met `NaN` pour les clients sans remise.

5.10 Étape 9 — Agréger (CA par catégorie)

```
# Agrégation par catégorie (proxy = 1re lettre de id_produit)
df_v9 = (
    df_v8
    .assign(cat_proxy=df_v8["id_produit"].astype(str).str[0])
    .groupby("cat_proxy", as_index=False)
    .agg(ca=("montant_ttc_n2", "sum"),
         quantite_totale=("quantite", "sum"),
         nb_lignes=("id_vente", "count"))
    .sort_values("ca", ascending=False)
)
print(df_v9)
```

- **groupby(..., as_index=False)** garde la clé de regroupement comme colonne (et non comme index).
- **agg** est plus flexible que **sum()** : on peut agréger plusieurs colonnes en une fois, avec des fonctions différentes.

5.11 Étape 10 — Pivoter / dépivoter

```
# Pivot : une colonne par catégorie
df_v10 = df_v9.pivot_table(
    index=None, # pas d'index : on garde une seule ligne (mars)
    columns="cat_proxy",
    values="ca",
    aggfunc="sum",
    fill_value=0,
)
print("pivot :", df_v10.shape) # (1, 9) : 1 ligne × 9 colonnes (mars a 1 mois)

# Pour le brut complet (12 mois), le pivot est (12, 9)
# (mesure m05_brut_pivot_lignes × m05_brut_pivot_colonnes)

# Dépivot : transformer en 2 colonnes (catégorie, valeur)
df_v10_depivot = df_v10.reset_index().melt(
    id_vars=None, # pas de variable id : on a une seule ligne
    var_name="Categorie",
    value_name="CA",
)
print("dépivot :", df_v10_depivot.shape) # (9, 2) : 9 lignes × 2 colonnes
```

- **pivot_table** avec **index=None** et **columns="..."** : on pivote une seule ligne.
- **melt** avec **id_vars=None** : on dépivot toutes les colonnes. C'est l'équivalent de **UNPIVOT** en SQL.

5.12 Étape 11 — Exporter la table propre

```
# Export CSV
df_v8.sort_values("id_vente").to_csv(
    "01_socle_donnees/data/projection/mars2025_pandas.csv", index=False
)

# Export Parquet (recommandé pour les tables > 100 000 lignes)
df_v8.sort_values("id_vente").to_parquet(
    "01_socle_donnees/data/projection/mars2025_pandas.parquet"
)
```

- **index=False** est crucial : sans lui, l'index du DataFrame devient la première colonne du CSV, et l'empreinte sha256 diffère de celle de DuckDB.
- **sort_values("id_vente")** : on trie à l'export, comme dans le format canonique du C01 §7.1.

5.13 Le notebook reproductible (.ipynb)

Le **notebook** est l'unité de livraison : un fichier `.ipynb` qui, ré-exécuté du début à la fin, produit la même table. Les règles :

- **Une cellule par étape conceptuelle** (import, conversion, coupe, tri, dédoublonnage, réparation, enrichissement, agrégation, export).
- **Le préambule `controle_python.py`** charge les variables partagées (`pd`, `np`, `stats (scipy)`, `RACINE`, `df`, `pop`). Les variables sont partagées entre les cellules, mais **chaque cellule doit être autonome** (ré-exécutable isolément).
- **L'export en fin de cellule** : chaque cellule qui produit un résultat exporte ce résultat (CSV ou variable globale). Sans ça, le notebook est volatile.
- **Les assertions en fin de cellule** : `assert len(df_v4) == 6884` (mesure `m05_fil-rouge_mars_lignes`). Sans assertion, une erreur silencieuse passe inaperçue.

5.14 Le diagnostic en 4 questions (le « Health Check » pandas)

Quand un notebook ne fait pas ce qu'il devrait, **4 questions** dans l'ordre :

1. **Le `dtype` est-il bon ?** `df.dtypes` — `montant_ttc_n2` doit être `float64` ou `Int64`, pas `object`.
2. **Le compteur d'échec est-il cohérent ?** `df_v3["montant_ttc_n2"].isna().sum()` doit rendre 0 après nettoyage (114 avant).
3. **La jointure utilise-t-elle la clé complète ?** L'argument `on=` doit être une **liste** de 3 colonnes (`["id_client", "annee", "mois"]`), pas une chaîne unique.
4. **L'agrégation est-elle cohérente ?** `df_v9["nb_lignes"].sum()` doit être proche de 6 798 (les 6 798 lignes agrégées par catégorie).

Conseil professionnel. Le **Health Check** en 4 questions est le premier réflexe à installer quand un notebook ne fait pas ce qu'il devrait. `dtype`, compteur, jointure, agrégation : 4 questions dans l'ordre, 90 % des bugs sont trouvés en moins de 5 minutes.

6. Exemple concret — le notebook `mars2025.ipynb` complet

Le notebook **complet** du fil rouge mars 2025. C'est l'équivalent pandas de la requête Power Query de C02 et de la requête SQL de C03. Les 8 cellules sont commentées ; chaque cellule est autonome.

```

# Cellule 1 – Préambule (exécuté en premier par controle_python.py)
import pandas as pd
import numpy as np

SRC = "01_socle_donnees/data/brut/ventes_brutes.csv"
RACINE = "/home/user/formation-data-bi"

# Cellule 2 – Import (règle : dtype=str, keep_default_na=False, encoding=utf-8-sig)
df = pd.read_csv(SRC, dtype=str, keep_default_na=False, encoding="utf-8-sig")
assert len(df) == 243_360, f"brut : {len(df)} lignes ≠ 243 360"

# Cellule 3 – Conversion de montant_ttc (avec compteur d'échec)
df = df.assign(
    montant_ttc_n=pd.to_numeric(
        df["montant_ttc"].str.replace(" FCFA", "", regex=False).str.replace(" ", "", regex=False),
        errors="coerce",
    ),
)
nb_vides = (df["montant_ttc"].str.strip() == "").sum()
nb_echec = df["montant_ttc_n"].isna().sum() - nb_vides
print("échec conversion :", nb_echec) # 4 211 sur le brut complet ; 114 sur mars
(m05_filrouge_mars_montants_texte)

# Cellule 4 – Coupe à la date (mars 2025)
mars = df[df["date_vente"].between("2025-03-01", "2025-03-31")].copy()
assert len(mars) == 6_884, f"mars : {len(mars)} lignes ≠ 6 884 (mesure m05_filrouge_mars_lignes)"

# Cellule 5 – Tri stable
mars = mars.sort_values(["date_vente", "id_vente"], kind="mergesort").reset_index(drop=True)

# Cellule 6 – Dédoublonnage sur la clé métier 5-colonnes
CLE = ["id_ticket", "id_produit", "quantite", "montant_ttc_n", "heure"]
mars = mars.drop_duplicates(CLE, keep="first").copy()
assert len(mars) == 6_798, f"après dédoublonnage : {len(mars)} ≠ 6 798 (6 884 - 86 ;
m05_filrouge_mars_doublons)"

# Cellule 7 – Réparation des dates transposées
d = pd.to_datetime(mars["date_vente"])
inco = d.dt.month != mars["mois"].astype(int)
print("dates transposées :", inco.sum()) # 23 sur mars (m05_filrouge_mars_dates_transposees)
ds = d[inco]
mars.loc[inco, "date_vente"] = (
    ds.dt.year.astype(str) + "-" + ds.dt.day.astype(str).str.zfill(2)
    + "-" + ds.dt.month.astype(str).str.zfill(2)
)

# Cellule 8 – Enrichissement par les remises (clé complète)
remises = pd.read_excel("01_socle_donnees/data/brut/remises_manuelles.xlsx",
                        engine="openpyxl")
remises_dedup = remises.drop_duplicates(["id_client", "annee", "mois"], keep="first")
mars = mars.merge(
    remises_dedup[["id_client", "annee", "mois", "remise_consentie_montant"]],
    on=["id_client", "annee", "mois"],
    how="left",
)
nb_enrichi = mars["remise_consentie_montant"].notna().sum()
assert nb_enrichi == 101, f"enrichies : {nb_enrichi} ≠ 101 (mesure
m05_filrouge_mars_lignes_enrichies)"

# Verdict
total = mars["montant_ttc_n"].sum()
assert total == 464_096_003, f"total : {total} ≠ 464 096 003 (mesure m05_filrouge_mars_total_ttc)"
print(f"mars : {len(mars)} lignes · total = {total:,} FCFA")

# Export (CSV + Parquet, trié par id_vente)
mars.sort_values("id_vente").to_csv(
    "01_socle_donnees/data/projection/mars2025_pandas.csv", index=False
)
mars.sort_values("id_vente").to_parquet(
    "01_socle_donnees/data/projection/mars2025_pandas.parquet"
)

```

Verdict attendu pour mars 2025 : - 6 884 lignes avant dédoublonnage (mesure `m05_filrouge_mars_lignes`) - 6 798 lignes après dédoublonnage (6 884 - 86 ; mesure `m05_filrouge_mars_doublons`) - Total `montant_ttc` = 464 096 003 FCFA (mesure `m05_filrouge_mars_total_ttc`) - 101 lignes enrichies (mesure `m05_filrouge_mars_lignes_enrichies`) - 23 dates transposées réparées (mesure `m05_filrouge_mars_dates_transposees`) - Compteur d'erreur de conversion après nettoyage = 0 (114 lignes avant nettoyage ; mesure `m05_filrouge_mars_montants_texte`)

Attention. Les **assertions** en fin de cellule sont la version bloc-notes du contrôleur `controle_python.py`. Si une assertion échoue, le notebook s'arrête — c'est la **différence** entre un notebook traçable et un notebook qui produit silencieusement une table fausse.

Attention. L'ordre des méthodes dans une chaîne pandas est important : `df.assign(...)` puis `df.sort_values(...)` puis `df.drop_duplicates(...)` est l'ordre canonique du module. Si on fait `df.drop_duplicates(...)` avant `df.assign(montant_ttc_n=...)`, le dédoublonnage se fait sur les colonnes non converties — `montant_ttc` (texte) au lieu de `montant_ttc_n` (entier) — et les doublons « 1 200 FCFA » vs 1200 ne sont pas détectés. C'est l'objet de la leçon « convertir avant de dédoubler » du C01 §5.1.

7. Démonstration pas à pas — le verdict : un fichier, trois moteurs, une table

Le **contrôle** que le notebook pandas produit la même table que la requête Power Query de C02 et la requête SQL de C03 est la **clé de voûte** du module. La méthode est la même que dans le C01 §7 : sérialiser la sortie au format canonique, trier par `id_vente`, hasher en sha256.

7.1 La sérialisation canonique en pandas

```
# bloc publié dans le contrôleur controle_python.py (extrait)
import hashlib

mars_sorted = mars.sort_values("id_vente")
lignes = []
for r in mars_sorted.itertuples(index=False):
    lignes.append("|".join("" if pd.isna(v) else str(v) for v in r))

empreinte = hashlib.sha256("\n".join(lignes).encode("utf-8")).hexdigest()
print("empreinte pandas :", empreinte)
# attendu : la même empreinte que DuckDB et Power Query
```

- Le **calcul d'empreinte** est identique à celui de C01 §7.2 et de C03 §7.2 : trier par `id_vente`, joindre les colonnes par `|`, hasher en sha256.
- L'empreinte attendue est **identique** à celle de DuckDB et Power Query — c'est la **preuve** que les trois moteurs produisent la même table.

7.2 Le verdict du projet M05.P (la mesure d'ouverture)

Sur le **dossier projet M05.P** (`03_exercices/dossier_M05/`, recalculé en pandas 2,2,3 et vérifié en DuckDB 1,5,5 — les deux moteurs ont la même empreinte) : - 120 000 lignes finales (mesure `m05p_lignes_final`) - Total `montant_ttc` = 7 145 910 735 FCFA (mesure `m05p_total_ttc_final`) - Empreinte sha256 = 7cce2d0c6253f739faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465 (mesure `m05p_empreinte_sha256`)

À retenir. Le verdict du projet M05.P est mesuré en pandas ET en DuckDB, et les deux moteurs donnent la **même** empreinte sha256. C'est la **preuve** que le pipeline est portable et déterministe.

7.3 La rejouabilité : avril et mai 2025

Le **même** notebook, exécuté sur avril et mai 2025 (changement de la borne supérieure du `between`) :

```
# pour avril : between("2025-04-01", "2025-04-30")
# pour mai   : between("2025-05-01", "2025-05-31")
```

Lignes attendues : 6 639 (avril, mesure `m05_filrouge_avril_lignes`) · 5 530 (mai, mesure `m05_filrouge_mai_lignes`).

8. Erreurs fréquentes

- **Importer en int64 par défaut.** `pd.read_csv(path)` devine `montant_ttc` en object (texte) et `id_vente` en int64. Les `id_vente = 0` (ventes comptoir) sont mal lus, et les montants texte passent en silence. La règle est toujours `dtype=str` à l'import, puis convertir.
- **Oublier `keep_default_na=False`.** pandas transforme "", NA, N/A en NaN, et le compteur d'échec est faussé. La règle est toujours `keep_default_na=False`.
- **Faire une affectation inplace** (`df["col"] = df["col"].fillna(...)`) au lieu de `df.assign(col=...)`. L'affectation modifie l'original et brise la reproductibilité.
- **Joindre sur la clé partielle** (`on="id_client"`) au lieu de la clé complète (`on=["id_client", "annee", "mois"]`). La jointure naïve rend 3 978 lignes au lieu de 101 sur mars (mesure `m05_filrouge_mars_jointure_naive`).
- **Faire `merge(..., how="inner")` par défaut.** Les 10 clients inconnus de mars (mesure `m05_filrouge_mars_clients_inconnus`) disparaissent silencieusement, et le total n'est plus celui du fichier d'entrée. La règle est toujours `how="left"` sauf cas exceptionnel.
- **Oublier `sort_values` avant `drop_duplicates`** ou utiliser `keep="last"` au lieu de `keep="first"`. Le dédoublonnage retire alors les **lignes d'origine** au lieu des copies.
- **Oublier `.loc[inco, "col"]` dans la réparation des transpositions.** `df["col"][inco] = ...` lève un `SettingWithCopyWarning`, et la modification peut ne pas être appliquée. La règle est toujours `df.loc[masque, "col"] = ...`.
- **Oublier `index=False` dans `to_csv`.** L'index du DataFrame devient la première colonne du CSV, et l'empreinte sha256 diffère de celle de DuckDB.
- **Croire que `SettingWithCopyWarning` est sans danger.** C'est un **signal** : pandas vous dit que la modification peut ne pas être appliquée. La règle est toujours `df = df.copy()` avant de filtrer, et `df.loc[masque, "col"] = ...` pour les modifications.
- **Oublier la graine sur un `sample()`.** `df.sample(n=1000)` sans `random_state=42` : deux exécutions donnent deux mesures différentes. La règle est toute opération aléatoire porte une graine, et la graine est documentée.

À retenir. Les 5 erreurs qui coûtent le plus cher en pandas : (1) import sans `dtype=str` ; (2) absence de compteur d'échec ; (3) jointure sur clé partielle ; (4) `merge` avec `how="inner"` par défaut ; (5) affectation inplace sur une vue. La règle unique : toujours expliciter l'option par défaut.

9. Bonnes pratiques professionnelles

- **Chaque transformation a un nom, en toutes lettres.** `df_v8` (la jointure) plutôt que `df2`. Sans nom, le notebook est illisible et **personne ne peut le maintenir** (règle R7 du §3 du plan M05).
- **Le compteur d'échec n'est jamais en commentaire.** C'est la ligne `print("échec conversion :", nb_echec)` qui rend 114 sur mars avant nettoyage, 0 après. Sans compteur, on ne sait pas si la conversion est complète.
- **Les transformations sont enchaînées par assign ou pipe, pas par affectation inplace.** L'enchaînement est **fonctionnel** : chaque étape prend un DataFrame, en rend un autre, et l'original est intact. C'est la version pandas des étapes Power Query et des WITH SQL.
- **Le notebook est exporté en CSV ou Parquet en fin de cellule.** Sans export, le notebook est volatile : le collègue ne peut pas réutiliser la table sans ré-exécuter le notebook.
- **La jointure utilise la clé complète,** documentée dans un commentaire au-dessus du `merge`. C'est la règle 4 du §3 du plan M05.
- **Le verdict est publié en clé `m05_*`.** Chaque chiffre cité dans le notebook (6 884 lignes, total 464 096 003 FCFA, 114 montants texte, 86 doublons, 101 enrichies, 23 transpositions) provient de `chiffres_manuel.py`.

Conseil professionnel. Le **Health Check** en 4 questions du §5.14 est le premier réflexe à installer quand un notebook ne fait pas ce qu'il devrait. `dtype`, compteur, jointure, agrégation : 4 questions dans l'ordre, 90 % des bugs sont trouvés en moins de 5 minutes.

9.1 Migration pandas 2.x → 3.x

Encadré « Migration ». Le module utilise pandas **2,2,3** dans l'atelier (règle 9 du §3 du plan M05). La migration vers pandas **3.x** est enseignée en M08. Les principales divergences à connaître :

- **`inplace=` est déprécié** dans pandas 3.x. La règle du module (toujours `assign` plutôt qu'affectation) est déjà conforme.
- **`DataFrame.iterrows()` est lent** : remplacé par `itertuples()` ou la vectorisation.
- **`df.append()` est supprimé** : remplacé par `pd.concat([df1, df2])`.
- **Le type `Int64` (nullable) est plus strict** : les `NaN` sont autorisés, mais pas les opérations implicites avec `int64`. La règle du module (`montant_ttc_n2` en `Int64`) est déjà conforme.

10. Exercice guidé — le notebook `avril2025.ipynb` paramétré (45 min, /10)

Objectif. Reprendre le notebook `mars2025.ipynb` du §6 et le rendre **réellement paramétré** sur le mois cible. Vérifier que le notebook produit avril 2025 avec les bons chiffres.

Énoncé.

1. Créer la constante en haut du notebook : `MOIS_CIBLE = 4` (avril).
2. Modifier la cellule 4 (coupe) pour utiliser `MOIS_CIBLE` : `between(f"2025-{{MOIS_CIBLE:02d}}-01", f"2025-{{MOIS_CIBLE:02d}}-{{28 if MOIS_CIBLE == 2 else 30}}")`. (Ou plus propre : un calcul de bornes par mois.)

3. Ré-exécuter le notebook du début à la fin. Vérifier : **6 639 lignes** (mesure `m05_filrouge_avril_lignes`).
4. Changer `MOIS_CIBLE = 5`, ré-exécuter, vérifier : **5 530 lignes** (mesure `m05_filrouge_mai_lignes`).

Barème (/10).

Critère	Points
La constante <code>MOIS_CIBLE</code> est créée	2
La cellule 4 utilise la constante (pas de date en dur)	2
Pour avril : 6 639 lignes	2
Pour mai : 5 530 lignes	2
Compteur d'erreur de conversion affiché dans le notebook (sa valeur n'est pas jugée, sa présence oui)	2
Total	10

Dans les faits. Cet exercice prend 35 minutes à un apprenant qui a déjà pratiqué le §6, et 1 h 15 à un apprenant qui découvre. La différence est presque entièrement sur l'étape 2 (calcul des bornes par mois) : un apprenant qui n'a pas compris que février a 28 ou 29 jours passe 30 minutes à chercher pourquoi `between("2025-02-01", "2025-02-30")` échoue.

11. Exercices autonomes

- **Exercice 11.1 (15 min).** Dans le notebook `mars2025.ipynb` du §6, remplacer la cellule 3 (conversion) par une conversion avec `errors="ignore"` (au lieu de `errors="coerce"`). Ré-exécuter et noter le compteur d'échec. C'est l'objet du piège n°1 du §8 : `errors="ignore"` laisse les cellules en texte, et le compteur est 0.
- **Exercice 11.2 (30 min).** Le notebook `mars2025.ipynb` produit 8 cellules. **Fusionner** les cellules 3 et 4 (conversion et coupe) en une seule. Vérifier que le compteur d'échec rend toujours 114 sur mars avant nettoyage.
- **Exercice 11.3 (45 min).** Reprendre le notebook `mars2025.ipynb` et **ajouter** la jointure naïve (`merge(..., on="id_client", how="inner")`) **dans une variable** `mars_naive`. Vérifier : **3 978 lignes** (mesure `m05_filrouge_mars_jointure_naive`), et comparer le total `remise_consentie_montant` de `mars_naive` avec celui de `mars` (la jointure naïve multi-compte les remises).
- **Exercice 11.4 (60 min, optionnel).** Mesurer le temps d'exécution du notebook `mars2025.ipynb` sur le brut complet (243 360 lignes). Répéter 5 fois, calculer la moyenne et l'écart-type. Comparer aux temps des requêtes Power Query (C02) et SQL DuckDB (C03) sur le même fichier. **Étiqueter** le résultat comme « estimation, à vérifier sur un fichier plus gros ».

12. Correction détaillée

- **Exercice 11.1.** `errors="ignore"` laisse les cellules non convertibles en texte. Le compteur d'échec est 0 (parce que `isna().sum()` ne compte que les NaN, pas les chaînes). C'est l'objet du piège n°1 du §8 : toujours `errors="coerce"` pour rendre les échecs visibles.

- **Exercice 11.2.** La fusion des cellules 3 et 4 est possible mais rend le notebook moins lisible. La règle du module est une cellule par étape conceptuelle — la lisibilité est prioritaire sur la concision.
- **Exercice 11.3.** La jointure naïve (`mars_naive`) rend bien **3 978 lignes** (mesure `m05_fil-rouge_mars_jointure_naive`). Le total `remise_consentie_montant` est **multiplié** par le nombre de mois couverts par le client — c’est l’erreur du débutant.
- **Exercice 11.4.** Les temps dépendent de la machine ; sur l’atelier de référence (pandas 2,2,3, machine virtuelle Linux 4 Go), les **mesures** publiées en C05 sont : 3,1 s pandas (mars), ≈ 1 min pandas (brut complet). Note : ces valeurs ne sont pas des clés `m05_*` ; elles sont étiquetées « estimation, à vérifier ».

13. Mini-projet M05.P4 — « Le notebook pandas du projet M05.P » (1 h 30)

Énoncé. Reprendre le notebook `mars2025.ipynb` du §6 et l’appliquer au **vrai fichier du projet** (`03_exercices/dossier_M05/ventes_2023_2024.csv`, 121 720 lignes). Le **livrable** est un notebook `projet2025.ipynb` qui : - Coupe à la date sur les **120 000 premières lignes** du fichier (la fenêtre du projet). - Applique toutes les étapes du §6. - Exporte la table finale en Parquet (`projet2025.parquet`). - Vérifie l’empreinte sha256 par rapport à l’ATTENDU du dossier (mesure `m05p_empreinte_sha256 = 7cce2d0c...`).

Critères de réussite.

1. **120 000 lignes** dans la sortie (mesure `m05p_lignes_final`).
2. **Total montant_ttc = 7 145 910 735 FCFA** (mesure `m05p_total_ttc_final`).
3. **493 dates transposées réparées** (mesure `m05p_dates_transposees_reparees`).
4. **Compteur d’erreur de conversion affiché** dans le notebook (sa valeur n’est pas jugée, sa présence oui).
5. **Empreinte sha256 identique** à l’ATTENDU (mesure `m05p_empreinte_sha256`).
6. **Aucune valeur de montant_ttc négative** n’a été filtrée silencieusement (les retours sont conservés).

Barème (/10). 2 points par critère sauf le 5 (empreinte) qui vaut 2 points, le tout sommant 10.

14. Boîte à outils du chapitre

Boîte à outils. Sept objets, un seul but — reproduire le notebook `mars2025.ipynb` en pandas :

1. **La table des 10 opérations en pandas** (§5) — l'application pandas de la table du C01. À imprimer.
2. **Le notebook `mars2025.ipynb` complet** (§6) — les 8 cellules, copiables et exécutables.
3. **Le compteur d'échec** (§5.3) — `df["col_n"].isna().sum() - (df["col"].str.strip() == "").sum()`, après chaque conversion.
4. **La jointure à la clé complète** (§5.9) — `merge(..., on=["id_client", "annee", "mois"], how="left")`, jamais sur `on="id_client"` seul.
5. **Le dédoublonnage sur clé métier** (§5.7) — `drop_duplicates(["id_ticket", "id_produit", "quantite", "montant_ttc_n", "heure"], keep="first")`.
6. **Le Health Check en 4 questions** (§5.14) — le diagnostic rapide d'un notebook qui ne fait pas ce qu'il devrait.
7. **Le verdict publié en clé `m05_*`** (§7) — chaque chiffre cité doit apparaître dans `chiffres_cites.json` ; sinon, c'est une promesse creuse.

15. Résumé du chapitre

- **pandas 2,2,3** est l'outil de **transformation en mémoire** pour l'analyste : plus lent que DuckDB (3,1 s vs 0,4 s sur 6 884 lignes), mais **plus riche en méthodes** (`pipe`, `assign`, `query`, `melt`, `pivot_table`, `merge`, `groupby`).
- La **chaîne de transformations** (`assign`, `pipe`) est l'équivalent pandas des étapes Power Query et des `WITH SQL` : chaque étape prend un `DataFrame`, en rend un autre, et l'original est intact.
- La **jointure à la clé complète** (`on=["id_client", "annee", "mois"]`) est l'unique protection contre la jointure naïve (3 978 vs 101 sur mars, 41 716 vs 1 714 sur la fenêtre).
- Le **compteur d'échec** (`isna().sum() - (cellules vides)`) est la première discipline : pandas `read_csv` est tolérant et avale les 1 710 montants texte sans un mot.
- Le **notebook reproductible** (`.ipynb` avec assertions en fin de cellule) est l'unité de livraison : un fichier qui, ré-exécuté du début à la fin, produit la même table.
- Le **verdict** du projet M05.P : **empreinte sha256 = 7cce2d0c6253f739-faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465** (mesure `m05p Empreinte_sha256`), identique entre pandas et DuckDB. C'est la **preuve** que le pipeline est portable.

16. À retenir

À retenir. pandas est un **moteur de transformation de tables** qui vit dans un notebook. La chaîne de transformations (`assign`, `pipe`) est l'équivalent des étapes Power Query et des `WITH SQL`. La jointure à la clé complète est l'unique protection contre l'erreur la plus coûteuse du module (3 978 lignes au lieu de 101 sur mars). Le notebook reproductible (assertions, export CSV/Parquet) est l'unité de livraison.

À retenir. Les 5 erreurs qui coûtent le plus cher en pandas : (1) `import` sans `dtype=str` ; (2) absence de compteur d'échec ; (3) jointure sur clé partielle ; (4) `merge` avec `how="inner"` par défaut ; (5) affectation inplace sur une vue. La règle unique : toujours expliciter l'option par défaut.

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 8 cellules du notebook mars2025.ipynb du §6 dans l'ordre. (Réponse : §6.)

Question 2. Quelle est la différence entre `dtype=str` et `dtype="str"` dans `pd.read_csv` ? Quand utilise-t-on chacun ? (Réponse : `dtype=str` force toutes les colonnes en texte ; `dtype={"col1": str, "col2": int}` force les colonnes spécifiques. On utilise `dtype=str` pour importer tout en texte, puis convertir colonne par colonne.)

Question 3. Pourquoi la **jointure à la clé complète** est-elle critique ? (Réponse : parce que la jointure naïve sur `id_client` seul rend 3 978 lignes au lieu de 101 sur mars (mesure `m05_filrouge_mars_jointure_naive`) — sans message d'erreur. La règle du module est toujours la clé complète, et on la documente dans un commentaire au-dessus du `merge`.)

Question 4. Qu'est-ce que le **dédoublonnage par rang** (`ROW_NUMBER() OVER ...`) en SQL devient en pandas ? (Réponse : `df.drop_duplicates(["id_ticket", "id_produit", "quantite", "montant_ttc_n", "heure"], keep="first")` après un `sort_values(["date_vente", "id_vente"], kind="mergesort")`. C'est l'équivalent pandas, plus court que la version SQL.)

Question 5. Sur mars 2025, combien de dates transposées sont réparées par la cellule 7 ? (Réponse : 23 lignes, mesure `m05_filrouge_mars_dates_transposees`.)

Question 6. Quelle est l'empreinte sha256 de la table `df_final` du projet M05.P ? Que prouve cette empreinte ? (Réponse : `7cce2d0c6253f739faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465` (mesure `m05p empreinte_sha256`). Elle prouve que la table produite par pandas est identique au bit près à celle produite par DuckDB — c'est la preuve que le pipeline est portable et déterministe.)

Question 7. Citez les 4 questions du Health Check pandas dans l'ordre. (Réponse : §5.14. (1) Le `dtype` est-il bon ? (2) Le compteur d'échec est-il cohérent ? (3) La jointure utilise-t-elle la clé complète ? (4) L'agrégation est-elle cohérente ?)